Error Mitigation in Noisy Quantum Circuits Workshop.

Student Lab Handout

Session Overview

Duration: 1h15m

Format: Paired coding.

Session Objectives

- Explain the difference between *quantum error correction* and *error mitigation*
- Apply measurement error mitigation using calibration circuits
- Implement Zero-Noise Extrapolation (ZNE)
- Diagnose when error mitigation succeeds and when it fails
- Develop intuition for accuracy vs. sampling tradeoffs on NISQ devices

Timed Agenda (75 minutes)

1. **0:00-0:05**: Why Error Mitigation?
2. **0:05-0:10**: Reference Noisy Circuit
3. **0:10-0:30**: Measurement Error Mitigation
4. **0:30-0:55**: Zero-Noise Extrapolation
5. **0:55-1:05**: When Mitigation Breaks
6. **1:05-1:15**: Wrap-up

1 Why Error Mitigation?

Near-term quantum devices operate in the *NISQ* (Noisy Intermediate-Scale Quantum) regime. In this regime:

- Qubits are noisy
- Gate fidelities are imperfect
- Long circuits are infeasible

Error Correction vs Error Mitigation

Quantum error correction:

- Requires logical encoding and ancilla qubits
- Preserves unitarity
- Enables arbitrarily long computations in principle
- Not yet practical on NISQ hardware

Error mitigation:

- Does *not* correct errors at the circuit level
- Operates on *expectation values*
- Trades circuit repetitions and classical post-processing for accuracy

Goal: estimate the noiseless expectation value $\langle O \rangle_0$ from noisy measurements $\langle O \rangle_\lambda$, without modifying the logical algorithm.

2 Reference Circuit with Noise

We study a two-qubit Bell state:

$$\left|\Phi^+\right\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

2.1 Ideal Circuit

Listing 1: Bell state preparation

```
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])
```

2.2 Noise Model

Two dominant noise sources are included:

- **Gate noise**: depolarizing error on CX gates
- **Readout noise**: probability of classical bit-flip during measurement

```
noise_model = NoiseModel()

# gate noise: Depolarizing gate error with 5% error rate on every two-qubit CX gate
noise_model.add_all_qubit_quantum_error(
    depolarizing_error(0.05, 2), ['cx'])

# readout error (e.g., 10-20% misread per qubit)
p0given1 = 0.15   # misread 1 as 0
#When the qubit is actually prepared (or ideally measured) as 1,
#the readout incorrectly reports 0 with probability 15%.
p1given0 = 0.15   # misread 0 as 1
readout = ReadoutError([[1-p1given0, p1given0], [p0given1, 1-p0given1]])

for qubit in range(2):
    noise_model.add_readout_error(readout, [qubit])
```

```
# 1. Initialize the simulator with the noise model
backend = AerSimulator(noise_model=noise_model)

# 2. Transpile and Run (Replaces execute)
transpiled_qc = transpile(qc, backend)
job = backend.run(transpiled_qc, shots=1024)

# 3. Get results and plot
noisy_counts = job.result().get_counts()
plot_histogram(noisy_counts)
```

This noisy result serves as the *unmitigated reference*.

3 Measurement Error Mitigation

Measurement errors are fundamentally different from gate errors, which is why they can often be mitigated effectively using classical post-processing techniques.

Measurement Errors Are Classical Once a quantum measurement is performed, the quantum state collapses, and the device outputs a *classical* bit string (e.g., 00, 01, 10, 11). Any error that occurs after this point is no longer quantum-mechanical.

Typical examples include:

- The detector reports `1` when the qubit was actually prepared in $|0\rangle$
- A voltage threshold is crossed incorrectly
- Classical electronics misinterpret the analog measurement signal

Key consequence:

- Measurement errors do *not* act on the density matrix ρ
- They act only on the classical probability distribution of measurement outcomes

For this reason, measurement error mitigation does not require modifying the quantum circuit or adding additional quantum gates.

Measurement Errors Are Stationary over Short Timescales Measurement errors are approximately *stationary* over the duration of a typical experiment. In practice, this means that:

- The probability of misreading $|0\rangle$ or $|1\rangle$ remains approximately constant
- Detector behavior does not drift significantly from shot to shot

This assumption is typically valid on timescales of:

- Minutes to hours on simulators
- Seconds to minutes on real quantum hardware

Because of this stability:

- We can measure the readout error once using calibration circuits
- The resulting calibration can be reused to mitigate many subsequent circuits

If readout errors varied rapidly in time, measurement mitigation would no longer be reliable.

Measurement Errors Are Empirically Calibratable Crucially, measurement mitigation does *not* require a physical model of the detector. Instead, we empirically characterize the measurement process by asking the device:

> When I know the correct answer, what do you report?

This is accomplished by running calibration circuits that prepare known computational basis states,

$$|00\rangle\,,\ |01\rangle\,,\ |10\rangle\,,\ |11\rangle\,,$$

and recording how often each state is measured incorrectly.
From this procedure, we extract empirical transition probabilities such as:

$$P(\text{measured } 1 \mid \text{true } 0),$$
$$P(\text{measured } 0 \mid \text{true } 1).$$

No assumptions are required beyond repeatability of the measurement process.

3.1 The Linear-Algebra Picture of Measurement Mitigation

Let:

- $\vec{p}_{\text{true}}$ denote the true probability vector of measurement outcomes
- $\vec{p}_{\text{measured}}$ denote the probability vector reported by the device

These vectors are related by a linear transformation:

$$\boxed{\vec{p}_{\text{measured}} = M\,\vec{p}_{\text{true}}} \tag{1}$$

where M is the *readout confusion matrix*.

3.2 Measurement Error Mitigation as Matrix Inversion

If the confusion matrix M is known and invertible, the true probability distribution can be estimated as:

$$\boxed{\vec{p}_{\text{mitigated}} = M^{-1}\vec{p}_{\text{measured}}}$$

Important points:

- This procedure is classical post-processing
- No additional quantum gates are required
- The underlying quantum algorithm is unchanged

This is why measurement error mitigation is:

- Computationally cheap
- Widely used in practice
- Extremely effective for many experiments

3.3 Why Negative Probabilities Appear

After inversion, the mitigated probability vector $\vec{p}_{\text{mitigated}}$ may contain negative components. This is not an error.

Negative values arise because:

- A noisy classical process is being mathematically undone
- Statistical fluctuations are amplified
- The result is a *quasi-probability distribution*

As long as expectation values of physical observables remain meaningful, this behavior is acceptable and expected.

```
from qiskit_experiments.library import CorrelatedReadoutError
# 1. Calibrate on the noisy backend
exp = CorrelatedReadoutError(physical_qubits=[0, 1])
exp_data = exp.run(backend=backend).block_for_results()

# 2. Extract Mitigator
#meas_fitter = exp_data.analysis_results("Correlated Readout Mitigator").value
# Get all results as a pandas DataFrame (avoids the warning)
df = exp_data.analysis_results(dataframe=True)
# Filter for the mitigator and extract its value
meas_fitter = df.loc[df["name"] == "Correlated Readout Mitigator", "value"].
    iloc[0]

# 3. Run the Bell Circuit
qc_t = transpile(qc, backend)
noisy_counts = backend.run(qc_t, shots=1024).result().get_counts()

# 4. Mitigate
mitigated_quasi = meas_fitter.quasi_probabilities(noisy_counts)
mitigated_dict = mitigated_quasi.binary_probabilities(num_bits=2)

# 5. Compare
plot_histogram([noisy_counts, mitigated_dict], legend=["Noisy", "Mitigated"])
```

Characterization (The Calibration Step) Here, the calibration is performed using the `CorrelatedReadoutError` experiment on qubits 0 and 1. This experiment executes $2^2 = 4$ calibration circuits, each preparing a different computational basis state: $|00\rangle, |01\rangle, |10\rangle$, and $|11\rangle$.

By measuring these states on the noisy backend (which has a 15% readout error per qubit), the experiment builds (M). Each entry $M_{y,x}$ represents the probability of measuring state y when state x was actually prepared. For example, the probability of preparing $|00\rangle$ but measuring $|01\rangle$ is recorded to account for the specific bit-flip noise in your simulator. The result of this process is the *Correlated Readout Mitigator*, which you extract from the `exp_data` dataframe to use for correction.

Execution and Mitigation Once the mitigator is "learned", you run the Bell state circuit (qc). However, the output is corrupted by two layers of noise:

- **Gate Errors:** The 5% depolarizing error on the CX gate physically alters the quantum state before measurement.
- **Readout Errors:** The 15% measurement error flips the bits during the final readout step.

The mitigation step `meas_fitter.quasi_probabilities(noisy_counts)` applies the inverse of the *confusion matrix* to your raw data. Mathematically, it calculates:

$$\vec{p}_{\text{mitigated}} = M^{-1}\vec{p}_{\text{noisy}} \tag{2}$$

In the final `plot_histogram`, you will see that the "Mitigated" bars for $|01\rangle$ and $|10\rangle$ are significantly lower than the "Noisy" ones because the readout-induced flips have been removed. However, these bars will not reach zero; the remaining counts represent the 5% CX gate error, which this specific mitigation technique cannot fix.

Key result: Measurement mitigation dramatically improves classical readout fidelity at low cost.

3.4 Why This Works So Well (and Why ZNE Is Harder)

Feature	Measurement Mitigation	ZNE
Acts on	Classical data	Quantum dynamics
Requires extra circuits	Yes (calibration only)	Yes (many folded circuits)
Noise amplification	Moderate	Large
Computational cost	Low	High

This explains why:

- Measurement mitigation is almost always applied in practice
- Zero-noise extrapolation must be used cautiously

4 Zero-Noise Extrapolation (ZNE)

Current quantum hardware operates in the so-called Noisy Intermediate-Scale Quantum (NISQ) regime, where gate imperfections, decoherence, and environmental noise significantly affect computation outcomes. As a result, the measured output of a quantum circuit generally deviates from the ideal, noise-free result.

Zero-Noise Extrapolation (ZNE) is a quantum error mitigation technique designed to estimate the result of an ideal noiseless computation *without* using quantum error-correcting codes. Instead of suppressing noise physically, ZNE removes its effects through post-processing.

The central idea of ZNE is simple:

> If the dependence of an observable on the noise strength is known or can be sampled, one can extrapolate the measured results back to the zero-noise limit.

In practice, this is achieved by deliberately increasing the noise in a controlled manner, measuring the corresponding outputs, and mathematically extrapolating these values to zero noise.

4.1 Procedure

The ZNE protocol proceeds as follows.

1. Baseline Execution The quantum circuit of interest is executed on hardware at its native noise level. An observable (for example, an expectation value or cost function) is measured and denoted as $E(\lambda)$, where λ represents the effective noise strength of the device.

2. Controlled Noise Amplification The same logical circuit is then executed multiple times with artificially increased noise levels. This is achieved without altering the ideal computation, using techniques such as:

- **Gate folding**: replacing a gate G with $GG^\dagger G$, which is logically equivalent but introduces additional noise;
- **Circuit stretching**: increasing gate durations, when supported by the hardware.

These executions yield measurements $E(2\lambda)$, $E(3\lambda)$, ... corresponding to amplified noise levels.

3. Noise-Response Modeling The measured values are plotted or fitted as a function of the noise-amplification factor. Empirically, the observable often varies smoothly with noise strength.

4. Extrapolation to Zero Noise A simple model (commonly linear or low-order polynomial) is fitted to the noisy data. Evaluating this model at zero noise yields an estimate of the ideal value: $E(0)$. This extrapolated value is taken as the mitigated result.

4.2 Noise Scaling via Gate Folding

Here we explain the implementation of circuit folding used to amplify noise in the context of Zero-Noise Extrapolation (ZNE). The goal of circuit folding is to increase the effective noise experienced by a quantum circuit while preserving its ideal (noise-free) logical action.

Conceptual Background Let U denote the unitary operator implemented by a quantum circuit without measurements. In the absence of noise, the ideal execution of the circuit applies U to an input state $|\psi\rangle$, producing

$$|\psi_{\text{out}}\rangle = U|\psi\rangle.$$

To amplify noise without changing the logical computation, ZNE uses the identity

$$UU^\dagger U = U,$$

where $U^\dagger$ denotes the Hermitian adjoint (conjugate transpose) of the unitary operator U. Because U is unitary, its Hermitian adjoint is equal to its inverse:

$$U^\dagger = U^{-1},$$

so that

$$UU^\dagger = U^\dagger U = I.$$

Although this identity holds exactly for ideal unitary evolution, each additional application of U and $U^\dagger$ introduces extra noise on real hardware. Repeating this construction increases the overall noise while leaving the noiseless transformation unchanged.

For an odd integer scale factor $s = 2k + 1$, the folded circuit implements

$$U_{\text{folded}} = U(U^\dagger U)^k,$$

which is logically equivalent to U but experiences approximately s times the noise.

Role in Zero-Noise Extrapolation The folded circuits generated by this function are executed at different scale factors $s = 1, 3, 5, \ldots$, producing noisy expectation values

$$E(s\lambda),$$

where λ represents the native noise strength.

These values are then used to extrapolate an estimate of the zero-noise result:

$$E(0) = \lim_{\lambda \to 0} E(\lambda).$$

```python
def fold_circuit(qc, scale_factor):
if scale_factor % 2 == 0:
    raise ValueError("Scale factor must be odd.")

# 1. Separate Unitary gates from Measurements
unitary_qc = qc.copy()
unitary_qc.remove_final_measurements() # Built-in Qiskit method

# 2. Perform the Folding (U -> U U* U)
folded = unitary_qc.copy()
num_pairs = (scale_factor - 1) // 2

u_inv = unitary_qc.inverse()
for _ in range(num_pairs):
    folded.compose(u_inv, inplace=True)
    folded.compose(unitary_qc, inplace=True)

# 3. Re-add measurements based on the original circuit's layout
folded.measure_all() # Adds new bits and measures everything

return folded
```

4.3 Compute Expectation Values

Expectation Value of the ZZ Observable for a Bell State To compute expectation values from measurement results, we make use of the *parity* of measured bitstrings.

The operator $Z \otimes Z$ (often written simply as ZZ) measures correlations between two qubits in the computational basis. Recall that the Pauli-Z operator acts on single-qubit basis states as

$$Z|0\rangle = +|0\rangle, \qquad Z|1\rangle = -|1\rangle.$$

When acting on a two-qubit computational basis state $|ij\rangle$, the ZZ operator acts as

$$(Z \otimes Z)|ij\rangle = (Z|i\rangle) \otimes (Z|j\rangle),$$

and the corresponding eigenvalue is the product of the two individual Z eigenvalues.

The Parity Rule The eigenvalue of ZZ depends only on the *parity* of the bitstring ij:

- If the number of 1s in the bitstring is even (0 or 2), the eigenvalue is $+1$.
- If the number of 1s is odd (1), the eigenvalue is -1.

Explicitly,

$$ZZ|00\rangle = +|00\rangle,$$
$$ZZ|11\rangle = +|11\rangle,$$
$$ZZ|01\rangle = -|01\rangle,$$
$$ZZ|10\rangle = -|10\rangle.$$

Thus, measuring ZZ is equivalent to assigning $+1$ to *even-parity* outcomes and -1 to *odd-parity* outcomes.

```
    def expectation_zz(counts):
    total_shots = sum(counts.values())
    exp = 0

    for bitstring, count in counts.items():
        # In Qiskit, bitstring '01' means qubit 1 is 0 and qubit 0 is 1.
        # For ZZ, we parity-check the bits.
        # Even number of 1s -> +1, Odd number of 1s -> -1
        parity = bitstring.count("1") % 2 # Checks if sum of bits is odd/even
        z_val = 1 if parity == 0 else -1 # Even -> +1, Odd -> -1

        exp += z_val * count # Accumulate weighted values

    return exp / total_shots
```

Expectation Value for the Bell State The circuit prepares the Bell state

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle).$$

Ideal Case

The expectation value of ZZ is given by the weighted average of the eigenvalues:

$$\langle ZZ\rangle = \sum_{ij} P(ij)\,\lambda_{ij},$$

where $P(ij)$ is the probability of measuring outcome $|ij\rangle$, and $\lambda_{ij} \in \{+1, -1\}$ is the corresponding eigenvalue of ZZ.

In the ideal (noise-free) case,

$$P(00) = \frac{1}{2}, \qquad P(11) = \frac{1}{2}, \qquad P(01) = P(10) = 0,$$

so the expectation value becomes

$$\langle ZZ\rangle_{\text{ideal}} = (+1)\left(\frac{1}{2} + \frac{1}{2}\right) = 1.$$

This reflects perfect correlation between the two qubits.

Noisy Case (ZNE Context)

In the presence of noise, gate errors and readout errors introduce population into the $|01\rangle$ and $|10\rangle$ states, which have odd parity and contribute negative eigenvalues.

As the noise is increased using circuit folding with scale factors 1, 3, and 5, the expectation value typically decreases. These noisy measurements are then used in Zero-Noise Extrapolation to estimate the zero-noise expectation value $\langle ZZ\rangle$.

```python
expectations = []

for s in scales:
    # 1. Fold the circuit (now works even with measure)
    folded_qc = fold_circuit(qc, s)

    # 2. Transpile with no optimization
    t_folded_qc = transpile(folded_qc, backend, optimization_level=0)

    # 3. Run
    job = backend.run(t_folded_qc, shots=1024)
    res = job.result()
    counts = res.get_counts()

    # 4. Compute expectation
    expectations.append(expectation_zz(counts))
```

```python
import matplotlib.pyplot as plt
import numpy as np

# 1. Fit the data
coeffs = np.polyfit(scales, expectations, deg=1)
zne_value = np.polyval(coeffs, 0)

# 2. Create the fit line (extending from 0 to your max scale)
x_fit = np.linspace(0, max(scales), 100)
y_fit = np.polyval(coeffs, x_fit)

# 3. Plotting
plt.scatter(scales, expectations, color='red', label='Measured Data')
plt.plot(x_fit, y_fit, '--', color='blue', label='Linear Fit')
plt.scatter(0, zne_value, color='green', marker='*', s=150, label=f'ZNE Value: {zne_value:.3f}')

plt.xlabel('Noise Scale (s)')
plt.ylabel('Expectation <ZZ>')
plt.title('Zero-Noise Extrapolation')
plt.legend()
plt.grid(True)
plt.show()
```

```python
print("Raw expectation:", expectations[0])
print("ZNE extrapolated:", zne_value)
```

4.4 Why Zero-Noise Extrapolation Works

For many near-term devices, noise effects scale approximately smoothly with circuit depth or execution time. By sampling the observable at multiple noise levels, ZNE infers this trend and removes the noise contribution mathematically.

Importantly, ZNE does not improve the physical quality of the hardware; instead, it corrects the measured data using classical post-processing.

4.5 Practical Advantages

Zero-Noise Extrapolation has several practical benefits:

- It requires no additional qubits.
- It does not rely on fault-tolerant error correction.

- It is compatible with existing quantum algorithms.

These features make ZNE particularly well suited for NISQ-era experiments.

4.6 Limitations

Despite its usefulness, ZNE has important limitations:

- It increases experimental cost, as multiple noisy circuit executions are required.
- Its accuracy depends on the assumption that noise scales smoothly.
- Excessive noise amplification can degrade the quality of extrapolation.

Consequently, ZNE must be applied carefully and interpreted as an approximation rather than an exact correction.

5 Wrap-Up

Technique Comparison

Technique	Best for	Cost	NISQ-ready
Measurement mitigation	Readout errors	Low	Yes
ZNE	Gate/decoherence	High shots	Yes
Error correction	Long circuits	Very high	No

6 Exercise: Exploring the Limits of Zero-Noise Extrapolation

In this exercise, you will use the Bell-state circuit as a testbed to explore when Zero-Noise Extrapolation (ZNE) improves results and when it fails. The goal is to develop intuition about the tradeoff between bias reduction and variance amplification.

6.1 Exercise Setup

We consider the Bell state

$$\left|\Phi^+\right\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle),$$

for which the ideal expectation value is

$$\langle Z \otimes Z \rangle_{\text{ideal}} = 1.$$

You will measure $\langle Z \otimes Z \rangle$ under increasing noise levels, apply ZNE using linear extrapolation, and compare the mitigated value to the unmitigated reference.

6.2 Part A: Baseline ZNE Performance

1. Use the depolarizing noise model with gate error probability $p = 0.05$ on the CX gate and readout error probabilities $p_{1|0} = p_{0|1} = 0.15$.
2. Choose noise scaling factors
$$s \in \{1, 3, 5\}$$
implemented via gate folding.
3. For each scale factor:

- Run the folded circuit with a fixed number of shots.
- Compute $\langle Z \otimes Z \rangle$ using the parity rule.

4. Perform a linear fit and extract the zero-noise value $\langle Z \otimes Z \rangle_0$.

Questions:

- How close is the raw (unmitigated) value to the ideal value of 1?
- Does ZNE move the result closer to the ideal value?
- How large are the statistical fluctuations across runs?

6.3 Part B: Increasing Gate Noise

Repeat Part A, but increase the depolarizing error probability on the CX gate to:

$$p = 0.10 \quad \text{and} \quad p = 0.20.$$

Questions:

- At what noise level does ZNE stop improving the estimate?
- Does the linear extrapolation become unstable?
- Do you observe extrapolated values outside the physical range $[-1, 1]$?

6.4 Part C: Increasing Folding Depth

Fix the gate noise at $p = 0.05$, but increase the set of scaling factors:

$$s \in \{1, 3, 5, 7\}.$$

Questions:

- Does adding more scale points improve or degrade the extrapolation?
- How does the variance of the extrapolated value change?
- Why does increased folding amplify statistical noise?

6.5 Part D: Reduced Shot Count

Repeat Part A using fewer shots (e.g. 256 or 128 shots).

Questions:

- How sensitive is ZNE to shot noise?
- Compare the raw and mitigated results under low statistics.
- Which source of uncertainty dominates?

6.6 Reflection

Summarize your findings in a few sentences. In particular, address the statement:

> "Error mitigation reduces systematic bias but amplifies statistical uncertainty."

Do your numerical results support this claim?

Reflection Questions

- Which mitigation technique was most effective?
- What limits mitigation in practice?
- When is it better *not* to mitigate?